PROGRAMACIÓN

UD 1: Introducción a la programación (II).

Índice

3. Estructura elemental de un programa Java.
4. Entornos Integrados de Desarrollo.
5. Variables.
6. Tipos de datos simples.
7. Declaración de variables.
8. Asignación de variables.
9. Listar variables.
10. Algunas cuestiones sintácticas.
11. Entrada y Salida elemental.
12. Operadores y expresiones.
13. Conversiones de tipos (CAST).
14. Algunas funciones predefinidas.
15. Bloques de instrucciones.
16. Documentación JAVA.
17. Consideraciones.

3. Estructura elemental de un programa Java.

- El programa más sencillo en JAVA: un único **bloque** principal llamado *main* que contiene, entre llaves, la secuencia de instrucciones a realizar.
  - Ejemplo:

```
public class HolaMundo
{
   public static void main (String args [])
   {
     System.out.println (“Hola Mundo…!!!”);
   }
}
```

3. Estructura elemental de un programa Java.

Recuerda:

1. Usa *System.out.println()* para escribir un texto en pantalla.
2. Todo método de Java lleva paréntesis.
3. Toda sentencia termina en punto y coma.
4. '\n' hace que el cursor salte al siguiente renglón.
5. Java es CASE SENSITIVE. Distingue mayúsculas de minúsculas.

3. Estructura elemental de un programa Java.

- **Actividad 1:** Modifica el '*HolaMundo.java*' usando el salto de línea, para mostrar tu nombre en una línea y tu apellido en otra.

- **Actividad 2:** Modifica el programa anterior para que escriba el nombre de 2 personas en dos líneas diferentes de la pantalla.

4. Entornos Integrados de Desarrollo.

- **Entorno Integrado de Desarrollo** (IDE) es una *herramienta con la cual poder desarrollar y probar proyectos en un lenguaje determinado*.

4. Entornos Integrados de Desarrollo.

- Existen distintos IDEs para trabajar con JAVA
  - **Eclipse.**
    - Desarrollado primeramente por IBM, aunque actualmente es libre mantenido por la Fundación Eclipse (www.eclipse.org).
    - Puede ser usado para otros lenguajes de programación instalando unos plugins.
    - Para todo tipo de proyectos.
  - **NetBeans.**
    - Es libre.
    - Para todo tipo de proyectos.
- **Cuestión**: *¿Es necesario un IDE para compilar y ejecutar JAVA?*

5. Variables.

- Una **variable** se puede representar como un *casillero en la memoria que guarda un valor.*
- **Variable:** Es una *zona de memoria donde se puede almacenar información del tipo que desee el programador*.
- Toda **variable** tiene un nombre o identificador, un tipo y un valor.

5. Variables.

- El **nombre o identificador de la variable** debe de cumplir ciertas reglas:
  - Deben empezar con una *letra del alfabeto inglés o guión bajo*.
  - Los siguientes caracteres pueden ser *dígitos, letras o guión bajo*. **Evita las letras con tildes y 'ñ'** (usa en su lugar 'ny').
  - No se deben usar palabras reservadas, éstas son aquellas propias del lenguaje Java como:
    - *public, void, static, boolean, float, int, class, true, false, null, for, if, else, extends …*
  - El estándar de Java sugiere utilizar **comenzar en minúscula e indicar cambio de palabra con mayúscula**. Ejemplo:
    - *nuevaVariable*
    - *cantPersonas*
    - *unNombreDeVariableMuyLargo*

5. Variables.

- **Declaración de una variable:** contiene el identificador de la variable y el tipo al que pertenece.
  - *TipoDeDatos IdentificadorVálido ;*
- **Tipo** al que pertenece una variable:
  - Define el *conjunto de valores que puede adquirir dicha variable*.
  - Y las operaciones que se pueden realizar con ella.
- **Asignación:** da valores a las variables o los reemplaza por otros.
  - *Identificador = Expresión;*
- Una **expresión** está formada por *operandos*, que pueden ser *constantes*, *variables* o *subexpresiones*, y *operadores*, y tiene un *tipo asignado*. En una instrucción de asignación, *los tipos de la variable y la expresión deben de ser compatibles*. Ejemplos:
  - `int var1, var2;`
  - `char ch1, ch2='u';`
  - `float f1=2.0, f2=3.0+f1;`

5. Variables.

- Las **variables miembro de una clase** (las veremos más adelante) *se inicializan por defecto*, mientras las **variables locales** (son las que vamos a utilizar desde esta unidad didáctica) *NO se inicializan por defecto*.
  - Las *variables numéricas* se inicializan a cero, los *caracteres* a 'o' y las *referencias a objetos y cadenas* a 'null'.
- En Java las variables no pueden declararse fuera de una clase.

```
public class suma {
     static int n1=50; //variable miembro de la clase
     public static void main (String args[]){
         int n2=30; //variables locales
     }
}
```

- **Visibilidad de las variables**: por regla general, en Java, *todas las variables que están dentro de un bloque (entre { y }) son visibles y existen dentro de dicho bloque*.

5. Variables.

- Las **variables miembro de una clase** (las veremos más adelante) son las variables de una clase en concreto. Ejemplo:

```
public class suma {
    int n1 = 50; //variable miembro de la clase
    public static void main (String args[]){
        int n2 = 30; //variables locales
    }
}
```

- En cambio las **variables locales** (son las que vamos a utilizar desde esta unidad didáctica) *son aquellas que se usan de una manera local a un bloque, método, …* Para nosotros, de momento, van a ser aquellas que usemos en los métodos:

```
public class suma {
    int n1=50; //variable miembro de la clase
    public static void main (String args[]){
        int n2 = 30; //variables locales
    }
}
```

5. Variables.

Tipo de dato	Valor predeterminado (para campos)
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000' (que es el cero 'o')
String (o cualquier objeto)	null
boolean	false

6. Tipos de datos simples.

- Es el ***tipo de información que puede almacenar una variable***.

- Java dispone nativamente de 8 tipos de datos los cuales los podemos agrupar en 4 categorías:

6. Tipos de datos simples.

Tabla 1.1. Tipos de datos simples

Tipo de datos	Información representada	Rango	Descripción
byte	Datos enteros	-128 ⟷ +127	Se utilizan 8 bits (1 byte) para almacenar el dato.
short	Datos enteros	-32768 ⟷ +32767	Dato de 16 bits de longitud (independientemente de la plataforma).
int	Datos enteros	-2147483648 ⟷ +2147483647	Dato de 32 bits de longitud (independientemente de la plataforma).
long	Datos enteros	-9223372036854775808 ⟷ +9223372036854775807	Dato de 64 bits de longitud (independientemente de la plataforma).
char	Datos enteros y caracteres	0 ⟷ 65535	Este rango es para representar números en unicode, los ASCII se representan con los valores del 0 al 127. ASCII es un subconjunto del juego de caracteres Unicode.
float	Datos en coma flotante de 32 bits	Precisión aproximada de 7 dígitos	Dato en coma flotante de 32 bits en formato IEEE 754 (1 bit de signo, 8 para el exponente y 24 para la mantisa).
double	Datos en coma flotante de 64 bits	Precisión aproximada de 16 dígitos	Dato en coma flotante de 64 bits en formato IEEE 754 (1 bit de signo, 11 para el exponente y 52 para la mantisa).
boolean	Valores booleanos	true/false	Utilizado para evaluar si el resultado de una expresión booleanas es verdadero (true) o falso(false).

6. Tipos de datos simples.

- Para las *cadenas de texto* (conjuntos de caracteres), usaremos un tipo de datos que NO es un tipo de datos simple. Es el tipo de datos **String**. Y el valor que queramos se pone entre comillas dobles “”. Un ejemplo de un String es el siguiente:

```
String nombre = “Javier Martín”;
```

6. Tipos de datos simples.

Datos enteros:

- Se usan para representar números enteros con signo, pero SIN DECIMALES.
- Hay cuatro tipos: *byte*, *short*, *int* y *long*.

Tipo	Tamaño	Valores
Byte	1Byte (8 bits)	-128 a 127
Short	2 Bytes (16 bits)	-32,768 a 32,767
Int	4 Bytes (32 bits)	-2,147,483,648 a 2,147,483,647
Long	8 Bytes (64 bits)	-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807

- Las comas NO son decimales, es para separar de tres en tres cifras.

6. Tipos de datos simples.

Datos en coma flotante:

- Se utilizan para representar números reales (CON DECIMALES).
- Existen dos tipos *float* y *double*.
  - *float*: utilizado para representar los números en precisión simple.
  - *double*: utilizado para representar los números en precisión doble.

Tipo	Tamaño	Valores
float	4 Bytes (32 bits)	+/- 3.4E+38F (6-7 dígitos importantes)
double	8 Bytes (64 bits)	+/- 1.8E+308 (15 dígitos importantes)

6. Tipos de datos simples.

Datos caracteres

- El tipo de dato ***char*** se usa para almacenar caracteres Unicode simples.
- Debido a que el conjunto de caracteres Unicode se compone de valores de 16 bits, el tipo de datos **char *se almacena en un entero sin signo de 16 bits.***
- Además de los tipos de datos nativos *existe una gran variedad de clases destinadas a la representación de diversos tipos de información*.
- Una de los más utilizados es el tipo de dato **String** que ***representa las cadenas de caracteres***.

6. Tipos de datos simples.

Datos booleanos

- El tipo de datos **boolean** ***sólo acepta dos posibles valores***: *true* o *false*.
- Es utilizado para la creación de variables de control.

7. Declaración de variables.

- ***Para poder utilizar una variable debemos declararla antes***.
- Es necesario "avisarle" a Java qué variables vamos a usar, para este proceso ***se requiere especificar el tipo de dato, nombre o id y opcionalmente un valor inicial***.

```
tipoDato nombreVariable = valorVariable;
```

- Ejemplos:
  - int i;
  - char letra='a', opc; //aquí estamos declarando dos variables *letra* y *opc*.
  - float num, res=7.5;
  - boolean existe= false;

7. Declaración de variables.

- ***En una <u>misma línea puedo declarar varias variables del mismo tipo</u> (nunca diferente) separadas por una coma (,)***, a algunas darles valor y a otras no.
  - <u>Ejemplo:</u>
    - int a, b; //Declaro dos variables de tipo enteras.
      - Equivalente es esto:
        - int a;
        - int b;
    - double c, d=20; //Declaro dos variables de tipo doble, c no tiene valor y d vale 20.
      - Equivalente es esto:
        - double c;
        - double d=20;
    - double a, char c; //**INCORRECTO**, en una misma línea sólo pueden haber datos del mismo tipo.

8. Asignación de variables.

- Es el **proceso de darle un valor a una variable, *el mismo puede ser asignado al momento de la declaración o posteriormente usando el signo de igual***.
  - Es importante aclarar que *a la hora de asignar un carácter* (tipo *char*), el valor *debe estar entre comillas simples* (‘z‘).
  - Y al asignar un *float debemos terminar el numero con una ‘f’* (3.1416f), si la omitimos el compilador entenderá que estamos asignando un *double*.
  - Ejemplos:
    - int i = 7;
    - double numDouble = 3.1416;
    - float numFloat =3.1416f;
    - char opc= ‘z’;

9. Listar variables.

La sentencia ***System.out.println()***, se puede usar para *desplegar el contenido de las variables*.

- Ejemplo:

```
1.  public class listarVariables {
2.          public static void main(String args []){
3.                  int a;
4.                  float pi;
5.                  char letra;
6.                  a= 7;
7.                  pi= 3.1416f;
8.                  letra = 'z';
9.                  System.out.println(" a vale " + a + ", pi vale " + pi + " y letra vale
    " + letra);
10.         }
11. }
```

9. Listar variables.

- Como se puede apreciar en la línea 9, al invocar el método, *se debe pasar como parámetro el texto entre comillas (“ ”), más el nombre de las variables a mostrar*.

- ***El símbolo de más (+) es utilizado para concatenar cadenas y variables.***

10. Algunas cuestiones sintácticas.

- Las **palabras que se usan en un programa** son *identificadores* y *palabras reservadas*.
- **Identificadores:** son *nombres que el programador da a variables, métodos y clases*. Debe comenzar por una letra o guion bajo (_) y a continuación cualquier combinación de letras, números, el carácter subrayado _ , y el signo de dólar $.
- **Las palabras reservadas** tienen un significado fijo en el lenguaje de programación y NO se pueden usar como identificadores.

abstract	default	if	private	throw
boolean	do	implements	protected	throws
break	double	import	public	transient
byte	else	instanceof	return	try
case	extends	int	short	void
catch	final	interface	static	volatile
char	finally	long	super	while
class	float	native	switch	
const	for	new	synchronized	
continue	goto	package	this	

10. Algunas cuestiones sintácticas.

La ***documentación necesaria para explicar las diferentes partes que componen un programa*** se puede incluir en el mismo a través de **comentarios**.

- Hay distintas maneras diferentes de poner comentarios en Java:
  - Se pueden utilizar los ***símbolos /* para abrir comentario y */ para cerrarlo***.
  - También se puede utilizar //, que hace que el compilador interprete todo lo que resta de la línea como comentario.

- Por ejemplo:

```
public class Hola {
   public static void main (String args[]) {
      /* Ahora va el saludo */
      System.out.println ("Hola a todos");
   }
}
```

O bien:

```
public class Hola {
   public static void main (String args[]) {
      // Ahora va el saludo
      System.out.println ("Hola a todos");
   }
}
```

10. Algunas cuestiones sintácticas.

En Java existen **tres tipos de comentarios**:

- **Comentario de párrafo.**

```
/* Esto es un comentario
que puede tener mas de una línea */
```

- **Comentario de línea.**

```
// Esto sirve para comentar una única línea
```

- **Comentario de documentación *(javadoc)*.**

```
/** Esto además de ser un comentario
sirve para que el programa javadoc documente
nuestro programa automáticamente */
```

Se puede leer más en http://www.upct.es/~orientap/ConvencionesCodigoJava.pdf

10. Algunas cuestiones sintácticas.

En Java existen **caracteres especiales** que tienen un papel de *separadores*. Son los siguientes:

- **Paréntesis** () : Contiene *listas de parámetros* en la definición y llamada a un método.
- **Llaves** {} : Sirven para englobar *bloques de código y para valores iniciales de vectores*.
- **Corchetes** [ ]: Sirven para la *declaración de vectores y para hacer referencias a elementos de los mismos*.
- **Punto y coma ;** : Separador de instrucciones.
- **Coma ,** : *Separador de identificadores del mismo tipo* en una declaración de variables, *separación de argumentos/parámetros en definición de métodos*.
- **Punto .** : *Separador de nombres de paquetes de subpaquetes y clases y para referenciar elementos de un objeto*.

11. Entrada y Salida elemental.

- Vamos a usar la clase ***Scanner*** que deriva del paquete **java.util**, la cual sirve para ***recuperar datos del teclado***. Se inicializa así:
  - `Scanner entrada = new Scanner(System.in);`

    ó
  - `Scanner entrada = new Scanner(String);`

Entrada estándar de Java

- Los métodos más interesantes y que vamos a usar de esta clase *Scanner* son:
  - **nextByte()**: Lee un tipo de dato *byte* y lo devuelve para ser almacenado en alguna variable.
  - **nextDouble()**: Lee un tipo de dato *double* y lo devuelve para ser almacenado en alguna variable.
  - **nextFloat()**: Lee un tipo de dato *float* y lo devuelve para ser almacenado en alguna variable.
  - **nextInt()**: Lee un tipo de dato *int* y lo devuelve para ser almacenado en alguna variable.
  - **next()**: Lee un tipo de dato *String* y lo devuelve para ser almacenado en alguna variable. Sólo lee un token o palabra. Lee hasta que encuentra el delimitador espacio en blanco.
  - **nextLine()**: Lee un tipo de dato *String* y lo devuelve para ser almacenado en alguna variable. Lee una línea entera (lee hasta que encuentra un \n).
  - **hasNext()**: Devuelve *true* si hay elementos por leer en la entrada estándar.
  - **useDelimitier()**: Por defecto considera cada palabra hasta que encuentra el delimitador espacio en blanco. *Con esta función podemos cambiar dicho delimitador por cualquier otro.*

11. Entrada y Salida elemental.

- Esta clase **Scanner** contiene muchos más métodos que podemos usar. Para saber más sobre ella consulta la API de dicha clase aquí.

11. Entrada y Salida elemental.

- Para poder utilizar la clase **Scanner** se pondrá al principio de la clase esta instrucción especial (para importar la librería):

```
import java.util.*; //Importa todas las clases del paquete util
O
import java.util.Scanner; //Importa solo la clase Scanner
```

- Ejemplo 1:

```
import java.util.*;

public class EntradaSalida {
    public static void main (String args[])
    {
        Scanner entradaTeclado = new Scanner(System.in);
        String nombre;
        System.out.println("Dime tu nombre:");
        nombre = entradaTeclado.nextLine();
        System.out.println("Tu nombre es: "+ nombre);
    }
}
```

11. Entrada y Salida elemental.

- **Actividad**: Realiza un programa donde pidas al usuario su nombre, edad y altura, y al final le muestres un mensaje como éste:

```
Los datos ingresados fueron:
  Nombre: nombreUsuario
  Edad: edadUsuario
  Altura: alturaUsuario
```

11. Entrada y Salida elemental.

- **Solución actividad:**

```
1.  import java.util.Scanner;
2.  public class usarScanner {
3.          public static void main(String args []){
4.                  int edad;
5.                  float altura;
6.                  String nombre;
7.                  Scanner input = new Scanner( System.in);
8.                  System.out.println("Ingrese su nombre");
9.                  nombre=input.next();
10.                 System.out.println("Ingrese su edad");
11.                 edad=input.nextInt();
12.                 System.out.println("Ingrese su altura");
13.                 altura=input.nextFloat();
14.                 System.out.println("\n Los datos ingresados fueron: \n Nombre:
    " + nombre + "\n                    Edad: " + edad + "\n Altura: " + altura );
15.         }
16. }
```

11. Entrada y Salida elemental.

- **Solución actividad**: ACLARACIONES
  - En la línea 7 del ejemplo se está instanciando un objeto de la clase *Scanner*, esto se limita a definir una variable con tipo de dato Scanner y con el uso del operador *new* (destinado a la reserva dinámica de memoria), se invoca al constructor de la clase (lo veremos en temas posteriores).
  - En este caso se le pasa como parámetro el **archivo de entrada del sistema *'System.in'***, (podría ser por ejemplo cualquier otro archivo con extensión txt, y otros).
  - Como se puede apreciar en la línea 9, 11 y 13 existen *métodos destinados a la obtención de información, definidos para todos los tipos primitivos y el tipo String*.

12. Operadores y expresiones

Tabla 1.3. Operadores aritméticos

Operador	Uso	Operación
+	A + B	Suma
-	A - B	Resta
*	A * B	Multiplicación
/	A / B	División
%	A % B	Módulo o resto de una división entera

En el siguiente ejemplo se puede observar la utilización de operadores aritméticos:

```
int  n1=2, n2;
n2=n1 * n1;       // n2=4
n2=n2-n1;         // n2=2
n2=n2+n1+15; // n2=19
n2=n2/n1;         // n2=9
n2=n2%n1;       // n2=1
```

Estos operadores se van a usar para realizar operaciones aritméticas (sumar, restar, multiplicar, dividir, módulo (resto de la división)).

12. Operadores y expresiones

Tabla 1.4. Operadores relacionales

Operador	Uso	Operación
<	A< B	A menor que B
>	A > B	A mayor que B
<=	A<= B	A menor o igual que B
>=	A>= B	A mayor o igual que B
!=	A!= B	A distinto que B
= =	A= = B	A igual que B

En el siguiente ejemplo se puede observar la utilización de operadores relacionales:

```
int  m=2, n=5;
boolean res;
res =m > n;//res=false
res =m < n;//res=true
res =m >= n;//res=false
res =m <= n;//res=true
res =m == n;//res=false
res =m != n;//res=true
```

Estos operadores se van a usar para evaluar expresiones a *cierto* o *falso*.

12. Operadores y expresiones

Tabla 1.5. Operadores lógicos

Operador	Uso	Operación
&& o &	A&& B o A&B	A AND B. El resultado será true si ambos operandos son true y false en caso contrario.
|| o |	A || B o A | B	A OR B. El resultado será false si ambos operandos son false y true en caso contrario.
!	!A	Not A. Si el operando es true el resultado es false y si el operando es false el resultado es true.
^	A ^ B	A XOR B. El resultado será true si un operando es true y el otro false, y false en caso contrario.

En el siguiente ejemplo se puede observar la utilización de operadores lógicos:

```
int  m=2, n=5;
boolean res;
res =m > n && m >= n;//res=false
res =!(m < n || m != n);//res=false
```

Estos operadores se van a usar para evaluar expresiones a *cierto* o *falso*.

12. Operadores y expresiones

- **Diferencia** entre los operadores lógicos && y &, y || y |:
  - Nosotros sólo vamos a usar los operadores && y ||.
  - La diferencia que existe es que los **&&** o **||** (*cortocircuitados*), evalúan la expresión lógica por partes, es decir, primero la parte izquierda y si esta es cierta, comprueba la de la derecha. En cambio los operadores **&** y **|** (*no cortocircuitados*), evalúan ambas partes a la vez. En la siguiente unidad didáctica se habla más sobre estos operadores.

12. Operadores y expresiones

Tabla 1.6. Operadores unitarios

Operador	Uso	Operación
~	~A	Complemento a 1 de A (no lo usaremos)
-	-A	Cambio de signo del operando
--	A--	Decremento de A
++	A++	Incremento de A
!	! A	Not A (ya visto)

En el siguiente ejemplo se puede observar la utilización de operadores unitarios:

```
int  m=2, n=5;
m++; //  m=3
n--;  //    n=4
```

12. Operadores y expresiones

- **<u>Operadores unitarios:</u>** ACLARACIÓN.
  - **x++**: es equivalente a *'x=x+1;'* Es decir primero incrementa y luego asigna el nuevo valor a 'x'. Lo mismo ocurre con 'x--', pero en vez de sumar se resta.
  - **++x**: También existe. Ahora mismo no tiene sentido usarlo, pero cuando veamos condiciones y bucles si que le veremos sentido usarlo. Lo mismo ocurre con '--x', pero en vez de sumar se resta.

12. Operadores y expresiones

Tabla 1.7. Operadores de bits (no los vamos a usar)

Operador	Uso	Operación
&	A & B	AND lógico. A AND B.
|	A | B	OR lógico. A OR B.
^	A ^ B	XOR lógico. A XOR B.
<<	A << B	Desplazamiento a la izquierda de A B bits rellenando con ceros por la derecha.
>>	A >> B	Desplazamiento a la derecha de A B bits rellenando con el BIT de signo por la izquierda.
>>>	A >>> B	Desplazamiento a la derecha de A B bits rellenando con ceros por la izquierda.

En el siguiente ejemplo se puede observar la utilización de operadores de bits:

```
int  num=5;
num = num << 1; //  num = 10, equivale a num = num * 2
num = num >> 1; //  num = 5, equivale a num = num / 2
```

12. Operadores y expresiones

Tabla 1.8. Operadores de asignación

Operador	Uso	Operación
=	A = B	Asignación. Operador ya visto.
*=	A *= B	Multiplicación y asignación. La operación A*=B equivale a A=A*B.
/=	A /= B	División y asignación. La operación A/=B equivale a A=A/B.
%=	A %= B	Módulo y asignación. La operación A%=B equivale a A=A%B.
+=	A += B	Suma y asignación. La operación A+=B equivale a A=A+B.
-=	A -= B	Resta y asignación. La operación A-=B equivale a A=A-B.

En el siguiente ejemplo se puede observar la utilización de operadores de asignación:

```
int  num=5;
num += 5; //  num = 10, equivale a num = num + 5
```

12. Operadores y expresiones

- **Precedencia de operadores.**
  - **Utiliza los paréntesis para controlar las operaciones** sin tener que depender de la precedencia y para dejar los programas más legibles.

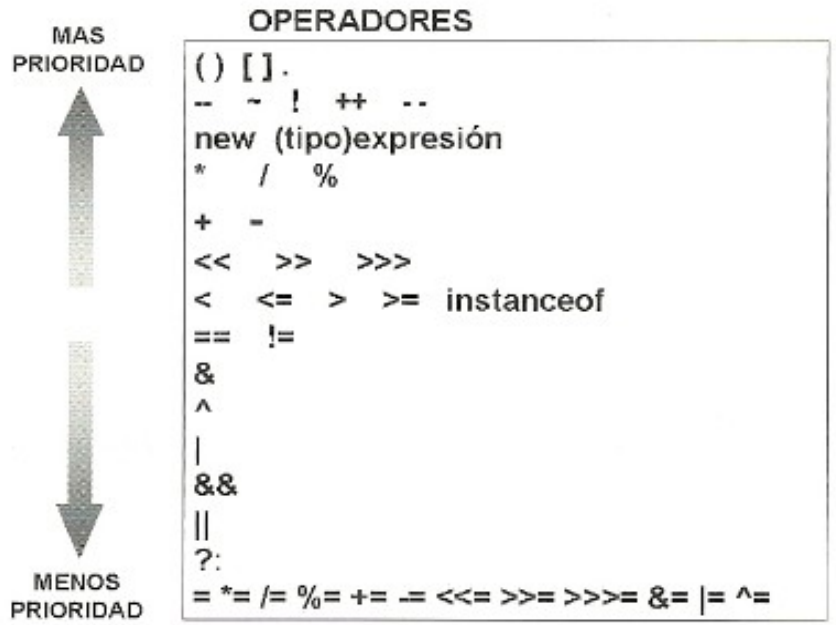


Figura 1.6. Prioridad de los operadores

Imaginemos que se tiene un código como el siguiente:

```
int a = 4;
a = 5 * a + 3;
```

12. Operadores y expresiones

- **Actividad 1:** Crear un programa que solicite un numero y calcule el doble del mismo.
  - El pseudocódigo sería el siguiente

    Inicio

    Mostrar "Ingrese un numero"

    Leer numero

    *doble=numero*2*

    Mostrar "el doble es" doble

    Fin

  - Ahora hazlo en Java!

12. Operadores y expresiones

- **Solución Actividad 1:** Crear un programa que solicite un numero y calcule el doble del mismo.

```
import java.util.Scanner;

public class doble {
        public static void main(String args []){
                 int num,resultado;
                Scanner input = new Scanner( System.in);
                System.out.println("Ingrese un número");
                num=input.nextInt();
                resultado=num*2;
                System.out.println("El doble del numero ingresado es:" + resultado  );
        }
}
```

12. Operadores y expresiones

- **Actividad 2:** Crear un programa que solicite dos números y retorne su suma.
  - Ejemplo:

    Ingrese un número: 8

    Ingrese otro número: 4

    La suma es 12

12. Operadores y expresiones

- **Solución Actividad 2:** Crear un programa que solicite dos números y retorne su suma.

```
import java.util.Scanner;


public class suma {
        public static void main(String args []){

                int num,num2;

                Scanner input = new Scanner( System.in);
                System.out.println("Ingrese un número");
                num=input.nextInt();
                System.out.println("Ingrese otro numero");
                num2=input.nextInt();
                System.out.println(num + "+" + num2 + "=" + (num+num2) );

        }
}
```

12. Operadores y expresiones

- **Actividad 3:** Crear un programa que solicite dos números enteros y retorne la división entre estos. Trabajar el concepto de división real (exacta) y entera (donde necesita resto o módulo).

  Observa la diferencia de almacenar el resultado en una variable de tipo coma flotante (double o float) y otra que no sea de coma flotante (por ejemplo int o long).

12. Operadores y expresiones

- **<u>Solución Actividad 3:</u>** Crear un programa que solicite dos números enteros y retorne la división entre estos.

```
import java.util.Scanner;


public class division {
	public static void main(String args []){
		int num,num2;
		float resultado;

		Scanner input = new Scanner( System.in);
		System.out.println("Ingrese un número");
		num=input.nextInt();
		System.out.println("Ingrese otro numero");
		num2=input.nextInt();
		resultado=(float)num/num2;
		System.out.println("La división entera entre " + num + " y " + num2 + " es " +
num/num2);
		System.out.println("El resto de la división entre " + num + " y " + num2 + " es
" + num%num2);
		System.out.println("La división real entre " + num + " y " + num2 + " es " +
resultado);

	}
}
```

12. Operadores y expresiones

- **Solución Actividad 3:** CONSIDERACIONES.
  - Si las variables num y num2 son de tipo *double*, resultado deberá ser *double*, ya que si es *int* dará error de compilación, al poder perder precisión en la operación. También tendrá que ser el módulo por el mismo motivo.
  - Si las variables num y num2 son de tipo *int*, y resultado de tipo *double*, se realizará la división entera (solo la parte entera) pero al ser *double* le añadirá un ".o" porque es una variable que lleva decimales, pero no es la división real o exacta.

13. Conversiones de tipos (CAST)

- Es importante destacar que a la hora de hacer una división real uno de los elementos de la misma debe ser de tipo *float* o *double*.
- Como en este caso ninguno de los dos lo era, se realizo una conversión de tipo (*cast*).
- El operador **cast** realiza este proceso, es decir, ***convierte datos, variables o expresiones a un nuevo tipo de dato.***
- Su formato general es (no siempre es así como veremos a continuación):

  (nuevo tipo) dato;

13. Conversiones de tipos (CAST)

- En una **asignación** *(identificador=expresion)*, *tanto la variable como la expresión deben ser del mismo tipo o de tipos compatibles*.
- **Conversiones implícitas:** Se realiza de *forma automática* entre dos tipos de datos diferentes. Requiere que la variable destino (izquierda) tenga más precisión que la variable origen (derecha).
  - Ejemplo:

```
byte dato1 = 3; short dato2 = 5;

dato2 = dato1;
```

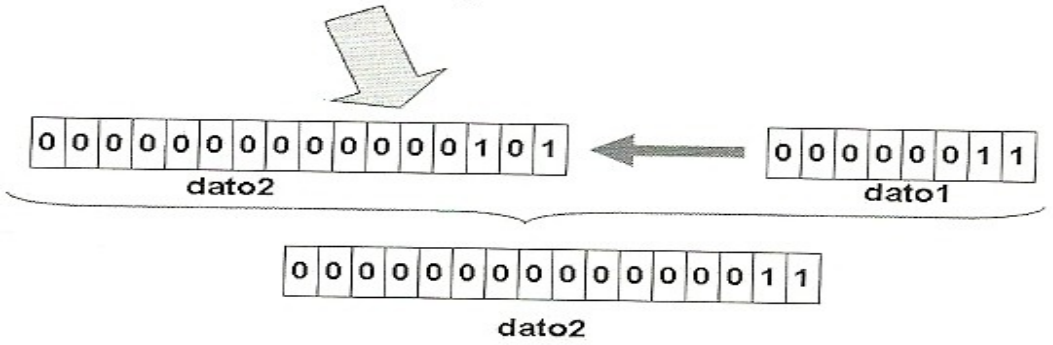


Figura 1.7. Ejemplo de conversión implícita

13. Conversiones de tipos (CAST)

- **Conversiones explícitas:** El programador ***fuerza la conversión*** mediante una operación llamada ***casting*** con el formato:

  (tipoDato) expresion

  - Ejemplo:

```
double dDato=5.5;
int iDato;
iDato = (int)dDato;
System.out.println(iDato); //sacará 5 por pantalla
```

  - *INTENTA a probarlo sin realizar el casting a int, es decir, quitando la parte (int).*

- NO se pueden realizar conversiones entre números y *booleanos* o reales y *booleanos*. *¿Por qué?* TIPOS INCOMPATIBLES!

13. Conversiones de tipos (CAST)

Conversiones explícitas: ASPECTOS A CONSIDERAR.

- Ejemplo 1:

```
double a=71.5;
int b= (int) a; //b valdrá 71, se desecha la parte decimal de a.
```

- Ejemplo 2:

```
double b= (int)(Math.round()*11);
```

b tomará un valor entre 0 y 10 (ya que Math.round() nunca nos devolverá un 1, con lo cual nunca tomará el valor 11, como máximo 10.99999999, que al hacer casting con un int se quitarán los decimales y al guardarlo en una variable de tipo double, se le añadirá un .0

- Ejemplo 3:

```
double a= 21.23; //Se separan los decimales por punto.                    //Leyendo de teclado por coma.
int b= a;
```

ERROR de compilación! Intentamos pasar un valor de un dato con mayor precisión a un dato con menor precisión: posible pérdida de precisión!

- Ejemplo 3:

```
int a= 21;
double b= a;
```

Va perfecto! El valor de b sería de 21.0, añade un decimal (un cero). Se puede realizar ya que el double tiene mayor precisión que el int.

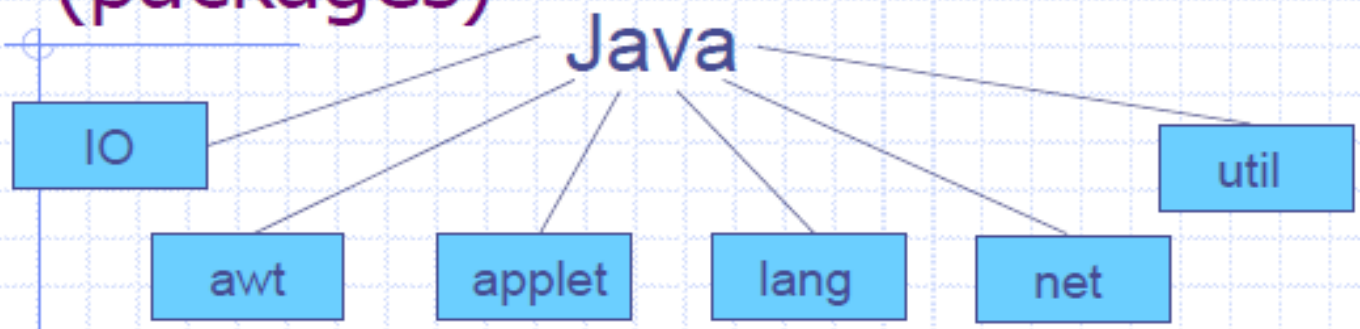
13. Conversiones de tipos (CAST)

Actividad: Declarar dos variables e1 y e2 de tipo entero e inicializarlas con 13 y 72, dos variables d1 y d2 de tipo double e inicializarlas con 13.69 y 69.13. A continuación mostrar por pantalla usando una sola línea de código:

1. El valor inicial que toma cada variable.
2. La suma de las variables enteras.
3. La resta de las variables enteras e1 – e2.
4. El producto de las variables enteras.
5. El cociente de las variables enteras e2 entre e1.
6. El resto de la división anterior.
7. La suma de las variables double.
8. La sustracción de las variables double.
9. El producto de las variables double.
10. La división de las variables double.
11. El resto de la división de las variables double.
12. El doble de cada variable.
13. La suma de todas las variables.
14. El producto de todas las variables.

14. Algunas funciones predefinidas.

Bibliotecas de clases de Java (packages)



- **Package lang**
  Clases con funcionalidades básicas, arrays, cadenas de caracteres, Entrada/salida, excepciones, hilos
- **Package util**
  Utilidades (números aleatorios, vectores, propiedades del sistema)
- **Package net**
  Conectividad y trabajo con redes: sockets, URL
- **Package applet**
  Desarrollo de aplicaciones ejecutables en navegadores
- **Package awt y swing**
  Desarrollo de interfaces gráficas de usuario
- **Etcétera, etcétera, ...**

14. Algunas funciones predefinidas.

- La clase **Math** define métodos para *calcular funciones trigonométricas, redondear valores reales y otros cálculos*.
  - Uso:

```
Math.nombreMetodo(…);
```

NOTA: Puedes *consultar todos sus métodos y variables* en la API de esta clase.

14. Algunas funciones predefinidas.

Algunas constantes y métodos útiles de la clase ***Math*** son:

1. Las constantes *E* y *PI*. Math.E=2.7182818284590452354, Math.PI=3.14.159265358979323846.

2. Las funciones de redondeo, con x de tipo *double*:

   - *ceil(x)* : devuelve el número entero más pequeño que es mayor o igual a x. Ejemplo: double x=879.327; double ceilx=Math.ceil(x); // ceilx=880
   - *floor(x)* : devuelve el número entero más grande que es menor o igual a x. Ejemplo: double x=879.327; double floorx=Math.floor(x); // floorx=879
   - *round(x)* : convierte el real x al entero más próximo. Ejemplo: int redondeo=Math.round(6243.45) // 6243; int redondeo=Math.round(6243.65); // 6244

3. Funciones trigonométricas:

   - *sin(x)* : calcula el seno del ángulo (en radianes) x
   - *cos(x)* : calcula el coseno del ángulo (en radianes) x
   - *asin(x)* : calcula el arco seno del ángulo x (x entre -1 y 1)
   - *acos(x)* : calcula el arco coseno del ángulo x (x entre -1 y 1)
   - *atan(x)* : calcula el arco tangente del ángulo x

14. Algunas funciones predefinidas.

ASPECTOS a considerar con los redondeos (casting y *Math.round()*):

- Ejemplo 1:

```
double a=71.5;
int b= (int)a; //b valdrá 71, se carga la parte decimal de a.
```

- Ejemplo 2:

```
double b= (int)(Math.random()*11);
```

b tomará un valor entre 0 y 10 (ya que Math.random() nunca nos devolverá un 1, con lo cual nunca tomará el valor 11, como máximo 10.99999999, que al hacer casting con un int se quitarán los decimales y al guardarlo en una variable de tipo doublé, se le añadirá un .0

- Ejemplo 3:

```
double b= Math.round((Math.random()*11));
```

b tomará un valor entre 0 y 11 (ya que Math.random() nunca nos devolverá un 1, con lo cual nunca tomará el valor 11, como máximo 10.99999999, que al usar Math.round() de ese número aleatorio lo redondeará hacia arriba o hacia abajo dependiendo el valor generado aleatoriamente.

14. Algunas funciones predefinidas.

- Algunas constantes y métodos útiles de la clase ***Math*** son (continuación):

4. Otras funciones:

   - *abs(x)* : calcula el valor absoluto de x (entero o real)
   - *exp(x)* : calcula e elevado a x (x es real)
   - *log(x)* : calcula el logaritmo natural de x (x real y no negativo)
   - *max(x,y)* : compara los números x e y (enteros o reales) y devuelve el mayor
   - *min(x,y)* : compara los números x e y (enteros o reales) y devuelve el menor
   - *pow(x,y)* : calcula x elevado a y. No está definida si x es negativo o 0 e y no es entero, ni tampoco si x=0 e y es negativo o 0
   - *random()* : genera un número (pseudo)aleatorio entre 0.0 y 1.0
   - *sqrt(x)* : calcula la raíz cuadrada positiva de x (x no negativo)

14. Algunas funciones predefinidas.

- Algunas constantes y métodos útiles de la clase ***Math*** son (continuación):
  - ***Math.sqrt(int x);***
    - x tiene que ser >=0. Si x es negativa, no hay soluciones reales, hay soluciones imaginarias, pero Java no las puede representar, entonces Java mostrará "*NaN*" *(Not a Number)*.

14. Algunas funciones predefinidas.

- **Actividad**: Realiza un programa que muestre por pantalla el resultado de las siguientes operaciones:

1. Cuánto es 3 elevado a 3?
2. ¿Cuánto vale la constante matemática e, uno de los números irracionales más importantes?
3. ¿Cuánto vale π (pi), que es la relación entre la longitud de una circunferencia y su diámetro en geometría euclidiana?
4. ¿Cuál es el seno de un ángulo de 90º?
5. ¿Cuánto vale la raíz cuadrada de 16?
6. Generar un número aleatorio con JAVA.
7. Dados los números 13 y 69, indicar cuál es el menor y cuál es el mayor.

15. Bloques de instrucciones.

- Un **bloque** es una *parte de un programa en Java englobada entre llaves y formada por declaraciones iniciales y una secuencia de instrucciones* (posterior a las declaraciones). SIEMPRE SE DECLARAN ANTES LAS VARIABLES Y LUEGO SE UTILIZAN.
- Ejemplo:

```
{
  Scanner entrada = new Scanner(System.in);
  int a = entrada.nextInt();
  System.out.println(a);
  entrada.close();
}
```

15. Bloques de instrucciones.

- Los **bloques** pueden declararse o *anidarse* unos dentro de otros.
  - ***Bloques externos*** (porque contienen a otros).
  - ***Bloques internos*** *(cuando están contenidos dentro de otros).*
- Las ***declaraciones* efectuadas al *comienzo de un bloque*** tienen sólo ***validez en el interior del mismo*** (incluso en bloques internos), ***fuera del bloque no surten efecto***.
- *Dentro de un bloque* se *pueden utilizar* tanto los *identificadores definidos en el mismo como en cualquier otro bloque externo que lo comprenda*.
- En *distintos bloques anidados NO se pueden definir identificadores con el mismo nombre*, en caso de hacerse así se generará un ERROR durante la compilación.

15. Bloques de instrucciones.

- Ejemplo bloques anidados:

```
{ int dia=10, mes=12, anyo=2000;
  {
   double temperatura;
   dia = 12;
   temperatura = 36.8;
   System.out.println(dia);
   System.out.println(temperatura);
  }
  System.out.println(dia);
  // la var. temperatura no se puede referenciar aquí
}
```

¿Qué valor de la variable '*dia*' se mostrará por pantalla para cada instrucción *System.out.println()*?

15. Bloques de instrucciones.

- Ejemplo bloques anidados:
  - ¿Qué valor de la variable '*dia*' se mostrará por pantalla para cada instrucción *System.out.println()*?
    - Solución: Ambas instrucciones *System.out.println()* mostrarán el valor de la variable día '12', ya que la variable *dia* es la misma para su bloque y el bloque interno, hace referencia a la misma variable o dirección de memoria.

15. Bloques de instrucciones.

- Se denomina **ámbito de una variable** a la *parte del programa en la que dicha variable es conocida y puede ser utilizada*.

- Una variable se dice que es **local** *en el bloque en el que se define* y **global** *para los bloques internos/anidados a éste*.

15. Bloques de instrucciones.

- Las siguientes **reglas** están relacionadas con el concepto de ***bloque y el uso de variables***:
  - Todas las ***variables*** *definidas en el* ***mismo bloque*** *(y en los bloques internos a éste) deben tener* ***nombres diferentes***.
  - Una *variable definida en un bloque es conocida desde esta definición hasta el final del bloque*. Como caso particular, *una variable definida en un bloque es conocida en todos los bloques internos a éste*.
  - Las *variables se deben definir al comienzo del bloque más interno en el que se utilizan*.

16. Documentación JAVA.

- Todos los paquetes, librerías, clases y funciones los podemos consultar en el **API de JAVA**, en nuestro caso para este curso usaremos la versión del JSE:
  - http://docs.oracle.com/javase/8/docs/api/

17. Consideraciones.

- Sólo se puede definir **una clase pública por fichero fuente**.
- Si **el fichero *.java* contiene una clase pública entonces *el fichero debe tomar el mismo nombre de la clase*** **(sin el .java)**.
- La ***compilación*** **correcta de un fichero *.java* *genera en el mismo directorio un fichero (bytecode)*** **llamado de la misma forma con extensión *.class***

18. Ejercicios.

1. Escribir un programa en Java que pregunte *Cómo te llamas?*, lea el nombre que introduces por teclado, por ejemplo *Albert* (acabando con NL) y escriba en pantalla *Hola Albert*.
2. Escribir un programa en Java que pregunte tu nombre, dirección y teléfono y escriba en pantalla una ficha.
3. Escribir un programa en Java que transforme en euros la cantidad que se introduce como dato en pesetas (1 euro son 166.386 pesetas).
4. Escribir un programa en Java que transforme una temperatura en grados Fahrenheit a grados Celsius *(Info: 0ºC==32F y c=(F-32)/1.8)*

18. Ejercicios.

5. Escribir un programa en Java para calcular la superficie y el volumen de una esfera a partir del valor del radio (supóngase que es un valor positivo).
6. Escribir un programa en Java para calcular el volumen de una esfera a partir del valor de su superficie (supóngase que es un valor positivo).
7. Una empresa de transporte por carretera ha adquirido vehículos nuevos que viajan más rápido que los antiguos. Les gustaría conocer cómo afectará esto a la duración de los viajes. Supóngase que la reducción media que se consigue del tiempo total de viaje es del 15%. Escribir un programa en Java que lea el horario de salida y llegada antiguo, calcule el nuevo horario de llegada y muestre en pantalla el nuevo tiempo de viaje y la nueva hora de llegada. Nota: Las horas se dan en el formato *hhmm*, un entero. Por ejemplo las 11:30, sería el entero 1130.

18. Ejercicios.

8. Realiza un programa que genere letras minúsculas aleatorias. Usa la tabla ASCII como ayuda y el método *random()* de la clase *Math*.
9. Ídem del anterior, pero que genere letras mayúsculas.
10. Ídem del anterior, pero que genere letras tanto mayúsculas como minúsculas. Evita que salgan los caracteres que no son letras que hay entre las letras mayúsculas y minúsculas.
11. Realiza un programa en Java que dados dos variables a y b cuyos valores se pedirán al usuario, intercambie los valores de a y b, y los muestre por pantalla.

18. Ejercicios.

12. Se necesita almacenar en una variable booleana el valor:
    - *Verdadero:* Si la *edad* es menor o igual a 28, el *nivel_de_estudios* es mayor que tres y los *ingresos* superan los 28.000 euros.
    - *Falso:* En caso contrario.

    Declara las variables enteras *edad, nivel_de_estudios* e *ingresos.*

    Pide los datos al usuario y comprueba su correcto funcionamiento.
13. Realiza un programa en Java que dada una variable **t** la cual contiene un tiempo en segundos, nos muestre dicho tiempo expresado en horas, minutos y segundos.

18. Ejercicios.

14. Realiza un programa en Java que dado un importe en euros, nos indique el mínimo número de billetes y la cantidad en monedas sobrante que se pueden utilizar para obtener dicha cantidad.